임베디드 시스템 소프트웨어
최종 프로젝트 보고서

이승우

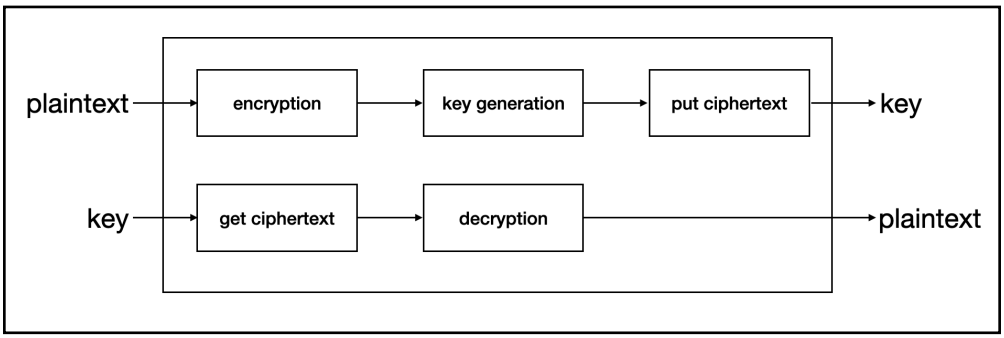
I. 개발 목표

- AES algorithm을 활용용한 텍스트 암호화 시스템 구현
- module programming
- android programming with JNI

II. 요약

1. 텍스트 암호화 시스템

(1) 4자리 숫자 key와 16바이트 텍스트로 구성된 key-value 데이터로 저장한다.
(2) 텍스트는 암호화되어 저장되고 key는 암호화된 텍스트를 기반으로 생성된다.
(3) key를 알아야만 텍스트를 읽을 수 있다.
(4) key를 알아야만 텍스트를 지울 수 있다.



overall flow of system

2. 구조

(1) operation
- user와 상호작용하며 insert / search / delete command를 받는다.

	입력	출력	동작
insert	텍스트	키	텍스트를 암호화해서 저장한다.
search	키	텍스트	암호화된 텍스트를 복호화해서 출력한다.
delete	키	-	텍스트를 지운다.

시스템 operation

(2) data unit
- text는 16바이트 문자열, key는 네자리 정수이다.

(3) data structure
- 데이터를 hash table 구조로 유지한다.

- key 사이즈는 9997 (가장 큰 네자리 소수) hash 사이즈는 97 (가장 큰 두자리 소수)이다.
- conflict 발생 시 linked list로 해결한다.
- node는 text 정보만 포함한다. - key를 별도로 저장하지 않는다.

III.개발 범위

1. AES

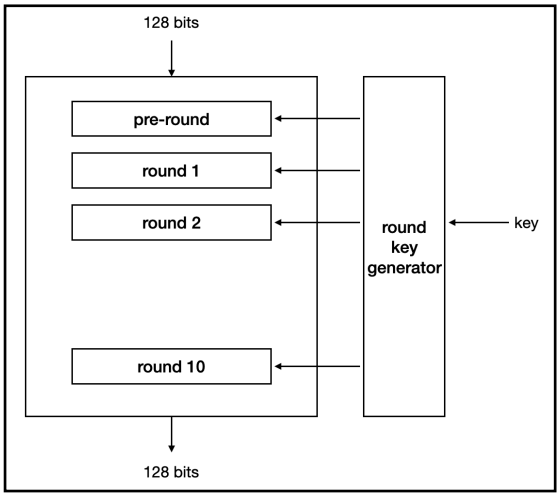
(1) 배경 지식

- AES는 cipher (encode, encrypt) 와 reverse-cipher (decode, decrypt) 에 같은 키가 사용되는 symmetric cipher의 한 종류로써 구현이 간단하고 빠르다는 특징이 있다. 따라서 하드웨어, 소프트웨어 어디서든 응용될 수 있다.
- AES의 종류로 key의 크기에 따라 AES-128, AES-192, AES-256 이 있고 본 과제에서 AES-128를 구현한다. 본 과제에서 AES의 설명은 AES-128를 기준으로 한다.
- AES의 데이터 단위는 state로 128 bits (16bytes)이며 state는 4 * 4 matrix로 취급되며 matrix elements는 1Byte 이다.

	0	1	2	3
0	12	38	94	83
1	45	4B	3C	3F
2	A1	21	54	84
3	BA	3B	19	10

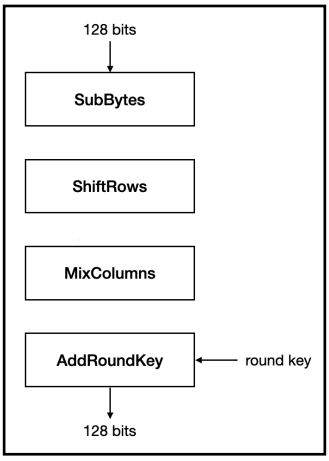
state 예시

- AES는 1개의 pre-round, 10개의 round로 구성되어 있으며 모든 round는 거의 동일한 구조로 설계되어 있다. (pre-round와 round 10 제외)



AES 구조

- 각 round는 SubBytes / ShiftRows / MixColumns / AddRoundKey 네 가지 함수가 차례대로 실행된다.



round 구조

- SubBytes는 substitution table를 참고해서 state의 모든 byte를 다른 byte로 substitute한다. 따라서 입력 state와 전혀 무관한 state가 출력된다. 필수적으로 predefined sbox (substitution table) 와 inv-sbox가 필요하다.
- ShiftRows는 state의 행을 left circular shift 한다. 0번째 행은 no shift, 1번째 행은 1 shift, 2번째 shift는 2 shift, 3번째 행은 3 shift 한다.
- MixColumn는 bit 단위 변환이 수행된다. 앞서 두 함수는 byte 단위의 변환인 반면 MixColumns은 bit 단위 변환이며 이웃 bit에 영향을 받는 변환이다. predefined constant matrix를 state의 column과 곱한다. (행렬 * 벡터) 곱하기 연산은 GF(2^8), 더하기는 GF(2)에서 이루어진다.
- AddRoundKey는 round key를 state의 각 column에 더한다.
- round key는 cipher key를 expansion해서 만들어진다. round key는 4개의 word로 표현되고 1 word는 4byte이다. pre-round의 round key는 cipher key 그 자체가 되고 이후로는 아래의 공식에 의해 round key가 생성된다.

$$
\begin{aligned}
w_i &= w_{i-1} + w_{i-4} && \text{when } i \neq 0 \mod 4 \\
w_i &= t_i + w_{i-4} && \text{when } i \equiv 0 \mod 4 \\
t_i &= \text{add}(\text{sub}(\text{rot}(w_{i-1}))) &&
\end{aligned}
$$

- t_i의 nonlinearity가 round key에 전파되어 key를 예측할 수 없도록 한다. add, sub, rot는 앞의 operation과 같은 함수들이다.

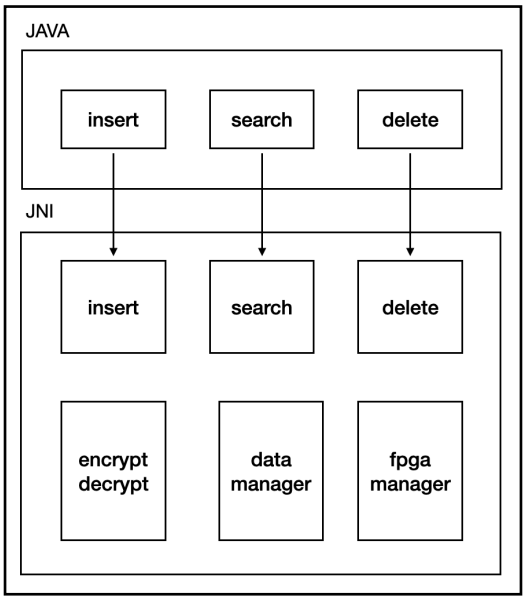
(2) 구현

- “introduction to cryptography and network security” 교재의 pseudo code를 참고해서 구현한다. 또한 sbox, constant matrix는 교재에 소개된 것을 활용한다.
- <unistd.h>에 system call로 삽입하여 JNI에서 encrypyion / decryption 할 수 있도록 한다.

2. android with JNI

(1) android

- MainActivity에 3개의 버튼이 사용자로부터 command를 받는 역할을 한다. 각각의 command는 다른 activity나 service를 통해 처리되고 다시 MainActivity로 돌아온다.
- android level에서는 fpga에 관여하지 않는다. - 오직 사용자 command만 처리하고 나머지는 jni layer에서 담당한다.



android-jni 구조

(2) JNI

- android layer에서 command를 받으면 해당하는 jni layer의 함수를 호출한다.
- jni layer의 함수를 두 종류로 나누어 생각한다. - 하나는 java에서 호출되는, 이름에 “JAVA_” prefix가 붙는 함수, 나머지는 helper 함수
- java에 의해 호출되는 함수들은 string 변환이나 java layer 함수 호출 같은 (*env)의 레퍼런스가 필요한 작업을 하고 나머지 작업은 helper 함수를 호출해서 처리한다.
- helper 함수는 encode / decode, device driver 활용, 데이터 관리를 맡는다.

3. module

(1) fpga

1) switch

- 사용자 입력 (key, text)을 받는다.
- 다음과 같이 abstraction 된다.

1	2	3	
4	5	6	
7	8	9	0

switch abstraction as number

abc	def	ghi	
jkl	mno	pqr	
stu	vwx	yz	next

switch abstraction as character

- 숫자 입력 시 fnd에 현재 입력이 표시된다. 문자 입력 시 lcd의 두번째 줄에 표시된다. 두 경우 모두 입력된 값이 취소되지 않는다.
- 문자 입력 시 next button이 눌려야 다음 문자를 입력할 수 있다.

2) lcd
- 첫번째 줄에 현재 command를 표시하고 두번째 줄에 결과를 출력한다.
- 다음과 같이 abstraction 된다.

current command
result

lcd abstraction

3) fnd
- key 값을 표시한다.

4) dot
- count down 9 -> 8 -> ⋯ -> 1 을 나타낸다.

(2) thread
- 두 가지 kernel thread를 활용하며 thread가 데이터를 만들 때 유저 process는 block된다.

1) number producer
- switch 입력을 네자리 숫자로 변환한다.
- 숫자가 업데이트될 때마다 fnd에 표시한다.

2) string producer
- switch 입력을 16바이트 문자열로 변환한다.
- 16바이트보다 작은 입력은 뒤에 padding byte을 넣어서 16바이트를 만든다.
- 문자열이 업데이트될 때마다 lcd에 표시한다.
- next button이 눌리면 dot을 전체 점멸한다. - reset button이 눌린 것을 표시하기 위해서

- 두 thread는 interrupt가 발생하면 종료된다.

(3) interrupt

- home button이 눌리면 interrupt handler가 trigger된다.
- interrupt handler의 top-half는 working thread가 있는지 확인하고 있다면 thread ID와 함께 button-half를 호출한다.
- bottom-half는 thread을 종료하고 user process을 wake up한다.

(4) timer

- user process는 sleep 대신 디바이스 드라이버의 wait_with_block 옵션의 ioctl를 호출한다.
- timer은 1초마다 호출되어 dot에 표시하며 그 동안 user process는 block된다.
- count가 0이 되면 user process을 깨운다.

IV.추진 일정 및 개발 방법

1. system 설계 (0610 ~ 0613)
   - system의 동작과 기능을 정의하고 데이터 단위를 정한다.
   - layer마다 요구사항을 정리한다.

2. AES 구현 (0614 ~ 0615)
   - application level에서 구현한다.
   - 정상적으로 동작하는지 확인하고 system call로 수정한다.

3. device driver 구현 (0616 ~ 0619)
   - 기본적인 구조를 설계하고 필수 기능을 구현한다.
   - 이후에 수정될 것을 감안하면서 개발한다.
   - android에서 활용되기 전에 test를 한다.

4. android 구현 (0620 ~ 0624)
   - 기본적인 ui와 activity, service의 구조를 설계한다.

5. jni 구현 (0621 ~ 0624)

6. device driver 수정 (0622 ~ 0624)

V. 결과

1. AES

(1) 호출방식

- ENCRYPTION의 경우 plain에 저장된 문자열을 encryption하여 cipher에 저장하고 DECRYPTION의 경우 cipher를 decryption하여 plain에 저장한다.
- 하나의 시스템 콜로 구현되어 flag를 통해 encryption / decryption을 결정할 수 있다.
- 다음과 같은 매크로를 정의해서 시스템 콜을 구현한다.

```
#include <linux/unistd.h>
#include <sys/syscall.h>

#define ENCRYPTION(plain, cipher) syscall(379, plain, cipher, 0)
#define DECRYPTION(plain, cipher) syscall(379, plain, cipher, 1)
```

system call 구현

2. module

(1) 기능

- 디바이스 드라이버는 다음과 같은 기능을 제공한다.

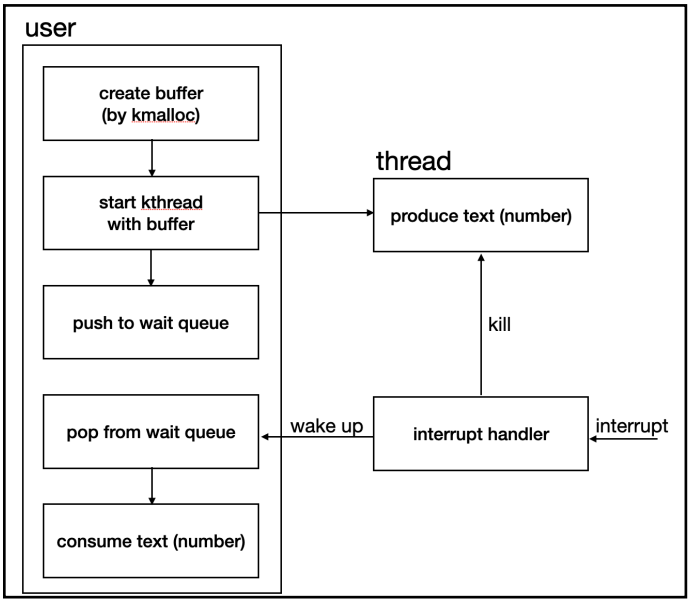
ioctl command	description
GET_TEXT	switch 입력으로부터 텍스트를 추출한다.
GET_NUMBER	switch 입력으로부터 4자리 정수를 추출한다.
SET_FND	fnd에 표시되는 숫자를 수정한다.
SET_DOT	dot에 표시되는 숫자를 수정한다.
SET_LCD_FIRST	text lcd의 첫번째 줄을 수정한다.
SET_LCD_SECOND	text lcd의 두번째 줄을 수정한다.
COUNT_WITH_BLOCK	user process을 최대 9초 간 block 한다.
DEVICE_CLEAR	모든 output device을 초기화한다.

(2) fpga function

fpga I/O function	description
int read_switch (void)	switch 입력을 숫자로 변환해서 반환한다. (reset은 0)
void set_fnd (int data)	integer -> short 로 변환해서 fnd에 출력한다.
void set_dot (int data)	data가 -1이면 clear, 10이면 full, 1~9이면 숫자 출력 fpga_dot_font.h에 정의된 배열을 참조한다.
void set_lcd (char *buf, int start, int length)	length만큼 buffer의 데이터를 length 만큼 lcd에 기록한다. start 변수에 따라 lcd에 출력될 위치가 결정된다.

(3) thread function

- ioctl(GET_TEXT) 일 때 버퍼를 생성해서 thread function을 시작하고 user process는 block된다. thread function은 switch로부터 텍스트(숫자)를 생성한다.
- thread가 종료되고 user process가 깨어나면 user process는 thread가 생성한 텍스트(숫자)를 유저 버퍼에 복사 후 메모리를 해제한다.
- kernel thread function은 숫자를 만드는 것과 문자열을 만드는 것, 두 가지가 있다. 두 thread는 동시에 실행될 수 없고 interrupt가 발생하면 종료된다.
- string producer은 padding byte로 '$'를 추가해서 문자열의 크기가 16byte가 되도록 보장한다.



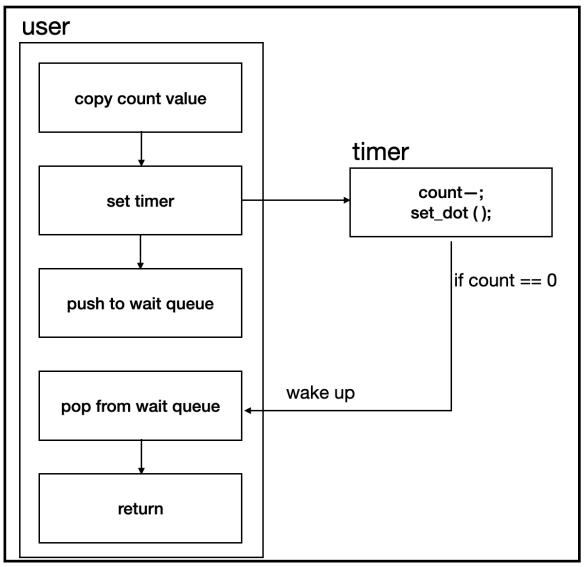
ioctl-GET_TEXT flow chart

(4) interrupt handler

- home button이 눌리면 invoke된다.
- working thread가 있으면 bottom-half을 스케줄링한다. bottom-half는 work queue로 하며 thread ID를 함께 넘겨주어 bottom-half에서 thread을 죽이고 user process을 깨운다.
- 첫 시도는 top-half에서 모든 일을 처리하는 형태였지만 top-half에서 thread을 죽이면 kernel panic의 원인이 된다. 따라서 bottom-half에서 처리하도록 변경했다.

(5) timer function

- 타이머 함수는 ioctl-COUNT_WITH_BLOCK 옵션을 통해 시작된다.
- 1초마다 호출되어 dot에 현재 count를 표시한다.



timer flow

3. JNI

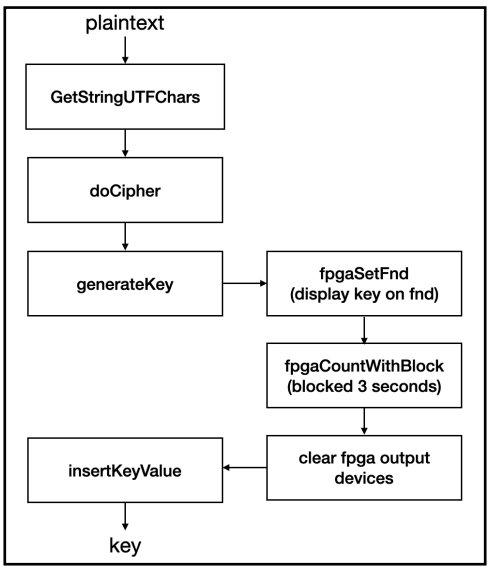
(1) JNI interface with JAVA layer

1) with MainActivity

- clearDevice: fpgaClearDevice 함수를 호출해서 fpga output device를 초기화한다.
- displayModeOnLcd: MainActivity의 getModeMsg method로 모드 메시지를 얻어 lcd의 첫번째 줄에 출력한다.
- createHashTable / destroyHashTable: hash table을 생성 / 파괴한다.

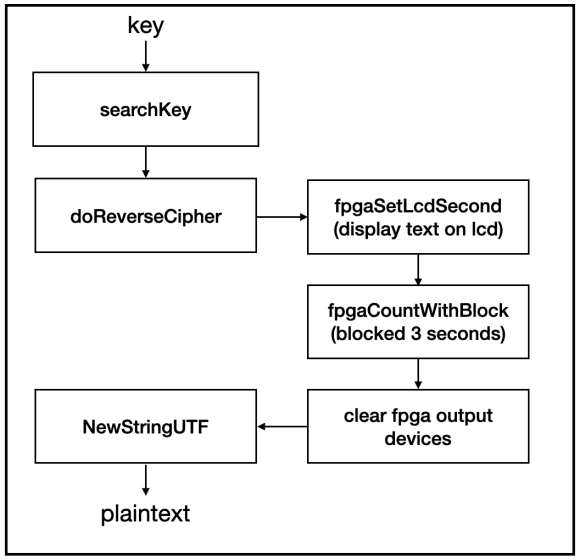
2) with InsertService

- getValueFromSwitch: fpgaGetText 함수를 호출해서 사용자 입력을 반환한다.
- encode: doCipher -> generateKey -> insertKeyValue 함수를 차례대로 호출해서 데이터를 암호화해서 저장한다. 그리고 key를 fnd에 3초간 출력하고 반환한다.
  fpgaCountWithBlock 함수로 fnd에 key가 표시되는 시간을 제한한다.



encode flow chart

3) with SearchService
   - getKeyFromSwitch: fpgaGetNumber 함수를 호출해서 사용자 입력을 반환한다.
   - decode: searchKey -> doReverseCipher -> fpgaCountWithBlock 함수를 차례대로 호출해서 decoding된 텍스트를 반환한다. key가 valid이면 텍스트를 lcd에 출력하며 invalid이면 “no such key”를 lcd에 출력한다. fpgaCountWithBlock 함수를 호출해서 검색 결과가 lcd에 3초 동안 보여지도록 한다.



decode flow chart

4) with DeleteService
   - getKeyFromSwitch: fpgaGetNumber 함수를 호출해서 사용자 입력을 반환한다.
   - deleteKeyValue: deleteKeyValue 함수를 호출해서 성공 / 실패에 따라 lcd에 메시지를 출력한다.

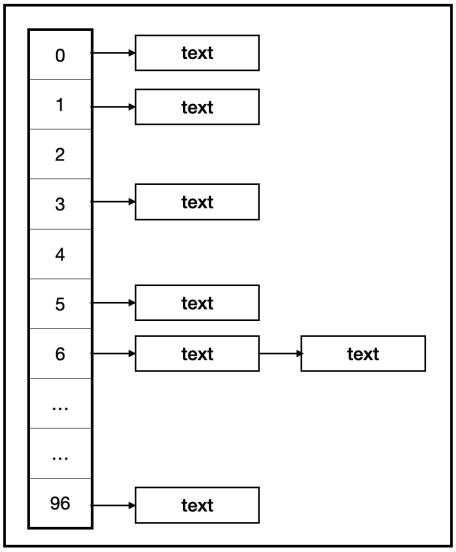
5) Semaphore

- initSema, waitSema, postSema, delSema 함수이며 sem_init, sem_wait, sem_pos, sem_destroy 함수의 wrapper function이다.
- 초기 개발에 background thread가 존재했기 때문에 구현된 interface이다.
- JAVA layer에서 직접 semaphore을 사용할 수 있다.

(2) data manager

1) data structure

- hash table 구조로 데이터를 관리한다.
- hash table의 크기는 97이고 key size는 9997이며 linked list로 collision을 해결한다.
- hash table은 암호화된 텍스트만 저장한다. 암호화된 텍스트를 그 자체로 unique한 key로 취급한다.



2) functions

a. generateKey

- 암호화된 텍스트의 character 값을 모두 더해 9997로 modulo 연산한다.
- generateKey를 통해 생성된 key가 데이터를 구별하는 식별자 역할을 한다.

b. insertKeyValue

- generateKey 함수에 의해 생성된 key 값을 97로 modulo 연산 후 hash table에 저장한다.

c. searchKey

- 사용자로부터 입력 받은 key를 97로 modulo 연산하고 해당하는 hash table의 데이터를 순회하면서 데이터의 key와 일치하는지 확인한다.
- 일치하는 텍스트를 발견하면 복사해서 반환한다.

d. deleteKey

- searchKey와 같이 동작한다. 다만 일치하는 텍스트를 삭제한다.

(3) fpga I/O manager

• 디바이스 드라이버를 사용하기 위한 wrapper function이다. 약간의 전처리 코드가 포함된다.

ioctl command	description
fpgaReadText	fpga device로부터 텍스트를 얻는다.
fpgaReadNumber	fpga device로부터 숫자를 얻는다.
fpgaSetFnd	fnd에 key를 출력한다.
fpgaSetDot	dot에 데이터를 출력한다.
fpgaSetLcdFirst	lcd의 첫번째 줄에 메시지를 출력한다.
fpgaSetLcdSecond	lcd의 두번째 줄에 데이터를 출력한다.
fpgaCountWithBlock	최대 9 초간 block된다.
fpgaClearDevice	fpga output device를 모두 초기화한다.

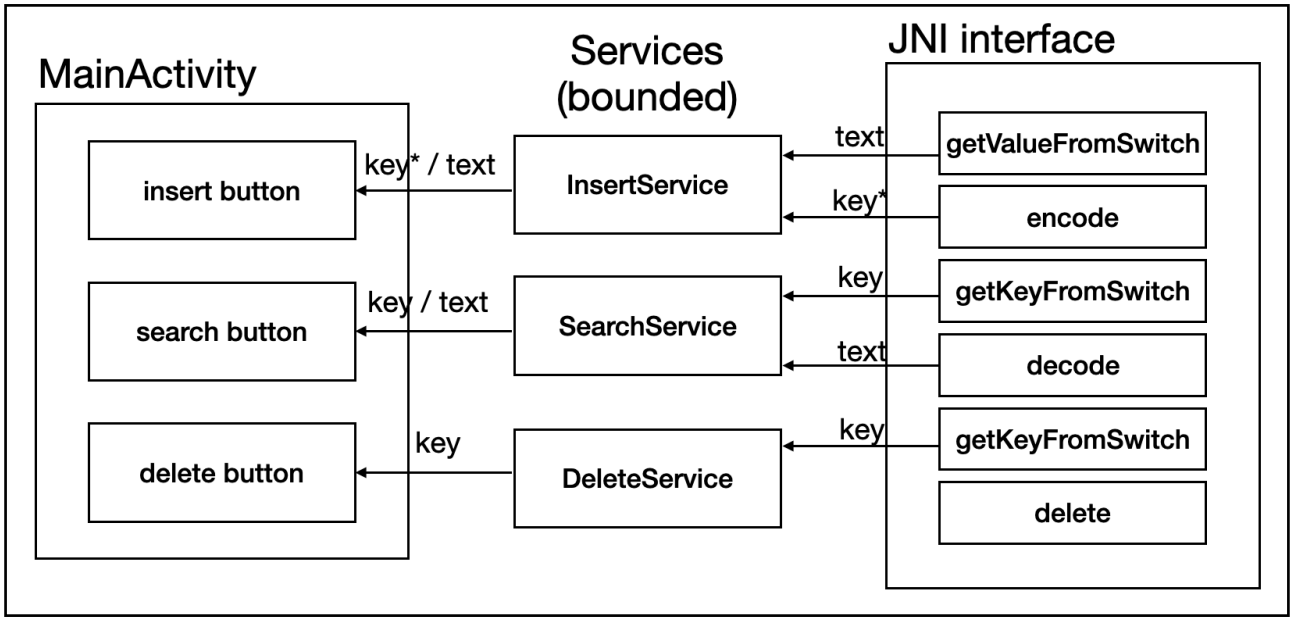
(4) encoding / decoding manager

• V-1-(1)의 ENCRYPTION / DECRYPTION system call을 호출해서 암호화 / 복화화를 담당한다.

4. android

(1) 안드로이드 구조

1) 구조



key: key from switch
key*: key generated by algorithm

- 1개의 Activity와 3개의 Service가 상호작용한다.
- Service는 하나의 command를 담당하며 native method을 호출한다.

2) MainActivity
- 유저로부터 command을 받아, command에 해당하는 service의 method로 command를 수행한다.
- lcd의 첫번째 줄의 메시지를 담당하여 사용자의 command가 있을 때마다 알맞은 메시지를 출력한다.
- 각종 초기화를 담당한다.
  - fpga device 및 jni layer의 hash table을 초기화한다.
  - button 등록 및 local service를 binding 한다.
  - Semaphore를 초기화한다.

3) InsertService
- JNI native method를 호출한다.
- 자료구조의 수정이 필요한 method 호출 시에 Semaphore로 lock을 얻고 호출한다.

```
Semaphore.waitSema( );
// call method
Semaphore.postSema( );
```

4) SearchService
- JNI native method를 호출한다.

5) DeleteService
- JNI native method를 호출한다.

6) Class Semaphore
- JNI layer의 semaphore 함수의 wrapper method가 정의되어 있으며 static method이므로 app의 모든 class에서 쓸 수 있다.

VI. 기타

1. 변경 사항들

(1) android
- 원래 android의 구조는 3개의 activity / 4개의 service로 구성되어 있었다.
- 여러가지 문제로 지금의 단순한 구조로 변경되었다.
  - encoding된 텍스트를 java layer에 올리면 발생하는 exception
  - 시스템이 단순해진 가장 큰 이유이다.

- 본래 계획은 encoding된 텍스트를 java layer의 activity와 service가 서로 주고 받으며 상호작용이 많은 시스템이었다.
- 하지만 encoding된 텍스트를 java layer에서 사용할 수 없어서 모든 코드가 JNI layer로 내려가고 JNI method가 줄어들었다.

- 3개 중 1개의 service는 background thread를 실행하는 서비스였다.
  - background thread의 역할은 다른 문제 (file system 실패) 로 모호해져서 탈락했다. semaphore도 필요없어졌다.

(2) file system

- 시스템의 다른 큰 축으로 파일 시스템을 구현하는 것이었는데 JNI layer에서 파일에 접근이 안되었다. 다른 우회 방법을 시도했지만 모두 실패했다.
- 현재 시스템은 메모리에 저장하는 방식이어서 하드웨어가 다운되면 데이터가 사라진다.
- 애초에 이런 문제를 인식해서 파일에 저장하는 방식, 또 background thread가 파일을 관리하는 구조를 설계했다.

2. 추후 구현

- AES system에서 cipher key는 중요하다. refresh가 되어야 한다. 하지만 본 과제에서 고정된 key로 암호화를 하다보니 같은 텍스트의 key값은 변하지 않는다.
- 이를 후반부에 인지했지만 system call부터 전부 고치기 부담스러워 넘어갔다.
- user로부터 cipher key를 입력 받아 같은 텍스트이어도 다르게 암호화되어 다른 key를 갖게 되는 시스템이 더 프로젝트의 방향에 알맞은 구조라고 생각된다. 현재 시스템은 찍어서 key를 맞춘다면 텍스트를 볼 수 있다.